Anomaly Detection Using Convolutional Autoencoder

Ben Kennedy
University of Victoria
Victoria, BC
benjaminkennedy@uvic.ca

Chase Westlake
University of Victoria
Victoria, BC
chasewestlake@gmail.com

Dylan Davis
University of Victoria
Victoria, BC
Dylandavis@uvic.ca

***Abstract*—This project explores anomaly detection using the Convolutional Autoencoder (CAE), which is an unsupervised method. The CAE has three stages; Encoding, Latent Space, and Decoding. It deconstructs images into reduced spatial dimensions and then reconstructs the image back to its original dimensionality. The anomalies are detected during the reconstruction in the decoding phase. The CAE generated very promising results during testing, demonstrating effectiveness of its implementation in anomaly detection. The updated Google Colab template can be found here.**

I. Introduction

Anomaly detection has significant real-world applications, including in medical image processing, industrial and commercial equipment monitoring, and manufacturing defect detection. The problem is determining what is considered normal, and then accurately detecting a deviation from the expected output.

Computer image analysis provides opportunities for effective detection techniques, which can cover discrete outputs, rather than relying on models which can only predict the likelihood of defects.

Convolutional Autoencoders provide a promising methodology for detection of anomalies with accuracy. The CAE compresses and input image through convolutional layers into a lower dimensional representation which captures essential features of the data. It then is reconstructed into its original dimensionality at the decoder stage, which attempts to recreate the image as close as possible to the original. While it is being reconstructed, if information deviates from the training, the reconstruction will be less accurate and can indicate errors as anomalies.

II. Literature Review

A. Literature Review

During the initial research to determine possible implementations, 3 were originally identified. Convolutional Neural Networks, One-Class Support Vector Machine, K-Nearest Neighbors, and Local Outlier Factor. In the end, Convolutional Autoencoders were selected.

B. Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are often used for anomaly detection in images because they can detect defects by learning similarity metrics or using autoencoders to reconstruct images and flagging reconstruction errors as anomalies. This allows for detection of defects without requiring any labeled anomaly data, which can be hard to have large sets of in industrial settings [1]

CNN's have been commonly found in anomaly detection systems; however, a study had found that CNNs may not be ideal due to difficulty in acquiring sufficient samples of defective systems in some operations. The issue is that due to the supervision necessary to train the models, the lacking datasets could result in undertraining. Undertraining is not a significant hindrance for CAEs.

C. One-Class Support Vector Machine

The One-Class Support Vector Machine (OCSVM) can be used for anomaly detection by using hyperplane-based detection. This method trains the model to recognize a normal pattern. If there is a discrepancy in the expected result, the model flags it as an anomaly. [2]

Relative to the autoencoder, OCSVM is simpler but does not achieve the accuracy in AUROC that an encoder can. In a medical study, OCSVM achieved an AUROC score between 0.5 and 0.84, but only in some cases. Otherwise, the score was

0.71 [3]. It is a better fast solution, but not ideal for robustness or complexity.

D. K-Nearest Neighbors

The k-Nearest Neighbors (k-NN) algorithm can be used for anomaly detection in images by pairing it with feature extraction and then identifying data points that are significantly farther away from normal samples. Given an image, the k-NN can compute the distance between the image's feature representations and its k closest neighbors from the training images. If the average distance exceeds some threshold, the image can be classified as anomalous [4].

K-NN can provide accurate comparisons of images for anomaly detection. However, relative to autoencoders, they cannot handle the degree of complexity that CAE's can. K-NN relies on Euclidean distance measurements [4], whereas the CAE can learn features and avoid the curse of dimensionality with this method in which it compresses images down into the latent space.

E. Local Outlier Factor (LOF)

LOF is a density–based technique that detects anomalies by comparing the local density of a point with those of its neighbors. If a point has a significantly lower density than its neighbors, it is flagged as an outlier. The local density is estimated by the typical distance a point can be reached by its neighbor [4].

LOF is like K-NN but preferred for anomaly detection. However, similar to K-NN it relies on Euclidean measurements for anomaly detection and cannot process higher dimensional data.

F. Benefits of Autoencoders

Autoencoders are unsupervised models that can handle complex data. They break down the image into a latent dimension, and do not need pre-labelling to determine when there are anomalies. Studies suggest that CAEs can provide robust anomaly detection especially when denoising, by using hidden activation layers to discretely remove noise on each layer [5].

III. Implemented Approach

The implemented approach for our anomaly detection system was a Convolutional Autoencoder (CAE) [5]. An example of a standard CAE can be seen in Figure 1. How it works is the CAE is trained on the good images in the training data iteratively until the difference of error between the reconstructed and input image is very low and then when anomalous test images are input, the reconstruction is poor and the error is high, indicating an anomaly. Our CAE consists of an encoder network which compresses the image into a lower-dimensional latent space (feature space) and a decoder network that does an inverse operation to attempt to reconstruct the original image from this latent space representation. A high-level representation of our entire system can be seen in Figure 2.

A. Model Architecture

Our CAE architecture consists of:

1. Encoder Network
   - Compresses the image through 3 convolution layers (Using channel sizes 3 to 16 to 32 to 256)
   - Uses ReLU activation function after each layer to introduce non-linearity
   - Latent dimension size of 256
2. Decoder Network:
   - Uses transposed convolutional layers to reconstruct the image from the latent representation (Examples of the latent representation can be seen in Figures 9 and 10).
   - Uses ReLU activation after each layer followed by a final sigmoid activation to scale outputs between 0 and 1

B. Loss Function

The error difference between the input and reconstructed image is calculated using the Mean Squared Error (MSE) formula which can be seen in Figure 3, where Xi is the original input pixel, Xi_hat is the reconstructed image pixel, and n is the total number of pixels. The error is squared to amplify the change it waits for larger relative errors while also ensuring that the difference is positive.

C. Training Process

Only normal (non-anomalous) images are used for training. The preprocessing consists of images being resized to 128x128 and being normalized to [0, 1] to keep it simple before training

begins. We did not use any data augmentation. For learning rate optimization we used the Adam optimizer, with a starting learning rate of 0.001, and found around 500 to 3000 epochs to be the most effective number of training cycles (see Figures 4 and 5), with a batch size of 8, and 256 latent dimension.

D. Evaluation Metrics

When optimizing the parameters, we focused on increasing AUROC score, accuracy, precision, F1-score, and recall. The AUROC score (Area Under the Receiver Operating Characteristic Curve) was our primary evaluation metric, as it measures the model's ability to distinguish between normal and anomalous instances across all possible thresholds. A score closer to 1.0 indicates a strong separation between good and defective reconstructions, while a score closer to 0.5 implies random guessing. This metric was especially useful because our model was trained in an unsupervised manner and could generalize without relying on labeled data during training.

Accuracy was used to determine the proportion of correctly identified anomalies and normal samples out of the total number of test cases. While useful, accuracy alone can be misleading in imbalanced datasets where the number of normal samples vastly outweighs the anomalies [6].

Precision and recall provided more nuanced views of our model's performance. Precision measured the percentage of detected anomalies that were actually defective, while recall quantified how many of the actual anomalies the model successfully detected. High precision indicates fewer false positives, and high recall indicates fewer false negatives. These are critical in industrial contexts where missing a defective part (low recall) or wrongly discarding a good product (low precision) could be costly [7].

The F1-score was used to combine both precision and recall into a single metric that balances both concerns. It is the mean of precision and recall and provides a comprehensive performance measure, especially when the data is imbalanced [8].

While our implementation emphasized unsupervised learning, we still evaluated these metrics by comparing the reconstruction error of test samples (both good and bad) against their known labels. This allowed us to create a Receiver Operating Characteristic curve and calculate the associated metrics using a range of MSE thresholds.

We also used error heatmaps as a qualitative evaluation tool, which helped visually identify which regions the autoencoder failed to reconstruct accurately. This was particularly useful when the model was producing high loss in non-anomalous background areas, prompting further investigation into model tuning and input normalization strategies.

By combining these metrics, we were able to form a holistic understanding of model performance and optimize our training parameters accordingly. Further analysis of these metrics across different epoch counts and latent dimensions is provided in the Results section.

E. Equations

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Figure 3: Loss Function Formula

IV. RESULTS & EVALUATION

The results as seen in figures 4 and 5 demonstrate the accuracy (AUROC) scores from the experiments. Multiple batch sizes, epochs, and latent dimensions were tested to evaluate optimal configurations.

It was found that batch size and latent dimensional space made little to no impact on the overall AUROC score, and that the iterations were critical in producing higher accuracy.

The maximum AUROC score was 86% for pasta, and 80% for the screws. As the epochs increase, the accuracy increases. However, after 3000 iterations for both pasta and screw datasets, there was a steep drop off in AUROC, that was accentuated with more epochs. Pasta seemed to be optimized at 500 epochs, whereas screws continued to improve until reaching 3000 epochs.

While both the pasta and screw datasets feature similarly shaped objects (primarily spiral and cylindrical), the model achieved slightly better performance on the screw's dataset. One plausible explanation is the higher consistency in texture, reflectivity, and background contrast within the screw images. Screws typically exhibit strong edges, metallic highlights, and clear separation from the background, which helps convolutional layers extract distinguishable features during both encoding and reconstruction.

In contrast, the pasta images present subtle color and lighting variations, with many pieces overlapping or occluding one another. These visual complexities may introduce noise during training and make it harder for the CAE to learn a consistent feature map. Additionally, the uniform matte texture and color of the pasta may reduce contrast in error maps, making it harder to distinguish anomalies, even if the reconstruction error is similar. This suggests that background separation and lighting uniformity may influence the model's anomaly detection performance more significantly than object shape alone.

A. Comparisons

Based on a CAE from a report in the Institution of Engineering and Technology [6], there was an accuracy score of 99.97% and an F1 score of 98.49%. Precision and Recall are 99.11% and 97.87% respectively.

The AUROC score we achieved is a maximum of 86% and at the 45th percentile threshold for pasta, F1 score, precision, recall, and accuracy was 84.21%, 88.89%, 80.00%, and 81.25% respectively.

For the 50th percentile threshold for screws, the F1 score, precision, recall, and accuracy was 88.89%, 100%, 80%, and 87.50% respectively, as seen in Table 1.

	Pasta	Screws
Accuracy	0.8125	0.8750
Precision	0.8889	1.0000
Recall	0.8000	0.8000
F1 Score	0.8421	0.8889
Threshold (%)	45	50

Table 1: Peak performance metrics for pasta and screws datasets

These scores are still considerably impressive considering that there is very little tuning, and no regularization [7]. There is a substantial amount of room for optimization to this CAE.

V. Conclusion

The autoencoder performed very well, despite having minimal tuning to performance-based systems. The encoder does not yet have regularization, and the optimizer has no tuning to its learning rate.

Future work could involve investigating superior alternatives to the Adam Optimizer, and what learning rate would best impact AUROC scores [9]. There is also room for experimenting with synthetically increasing the datasets by using transformations, which can be achieved by mirroring and flipping images to increase the complexity within the training [10].

More research and experimentation would need to be done on optimal implementations of regularization, but the methods expected to function well are of 3 choices; L1 or L2 regularization, and denoising [11].

Overall, the methods worked well. The number of epochs was optimized based on the current configurations, and the CAE receives strong metrics along accuracy, F1, precision and recall when compared with strong competitors [1].

The scores are reflective of a non-optimized model, with plenty of room to improve. With the proper tuning and implementation of regularization, and other techniques for model improvement, this CAE has potential for significant improvement, possibly near 100%.

It is important to note that the comparative scores listed are non-deterministic and can change based on the evaluations at the time of the testing or usage of the CAE.

Sets of screws and pasta. Prior to selecting the CAE, five other methodologies were examined to determine which options were most suitable for this application [6].

Convolutional Neural Networks, One-Class Support Vector Machine, K-Nearest Neighbors, and Local Outlier Factor were among the methodologies considered. They are all viable, but early on it was determined that CAE was the best choice, due to its ability to function as an unsupervised method while also being capable of handling higher dimensional datasets relatively easily.

CAE involves receiving input data and encoding it into the latent space, where it is then decoded and evaluated for differences in the training data. Our implementation was able to achieve notably high scores with minimal optimization outside of the basic implementation of the approach. With additional tuning and optimization, we likely could see metrics closing in on 100%.

VI. Figures and Tables

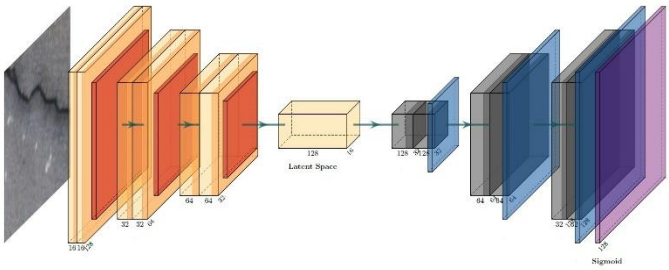

Figure 1: Convolutional Autoencoder example [10]

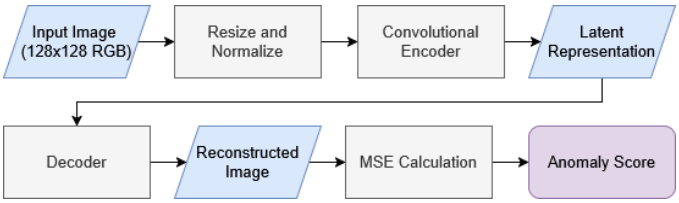

Figure 2: Our Anomaly Detection System Overview

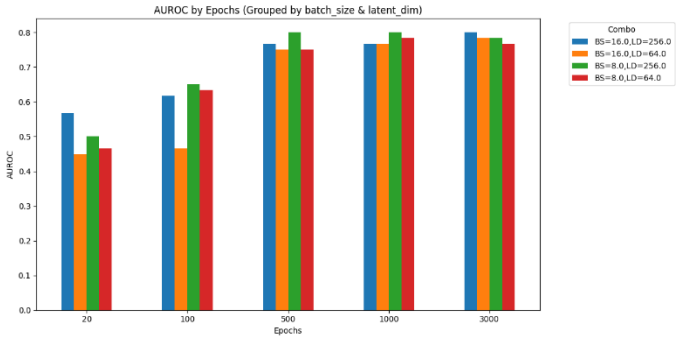

Figure 4: AUROC scores by Epochs for pasta dataset

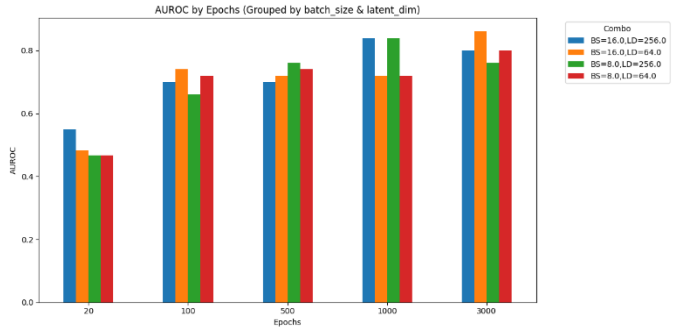

Figure 5: AUROC scores by Epochs for screws dataset

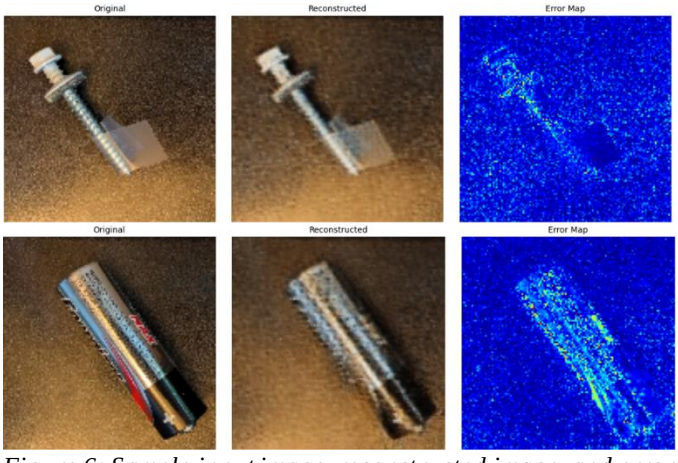

Figure 6: Sample input image, reconstructed image, and error heatmap

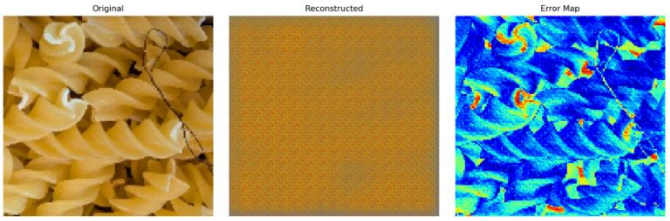

Figure 7: Sample anomalous image, reconstructed image, and error heatmap during early testing with low epoch

Figure 8: Sample anomalous image, reconstructed image, and error heatmap during early testing with low epoch

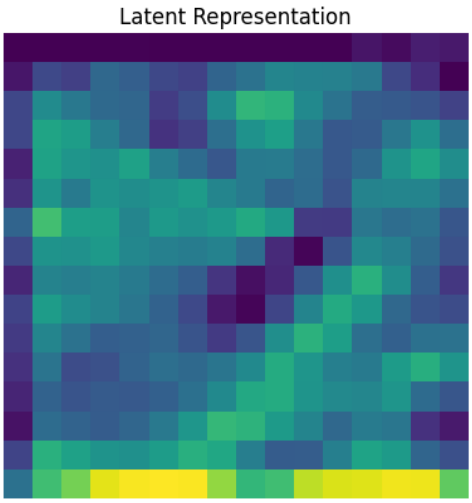

Figure 9: Channel 0 of latent representation

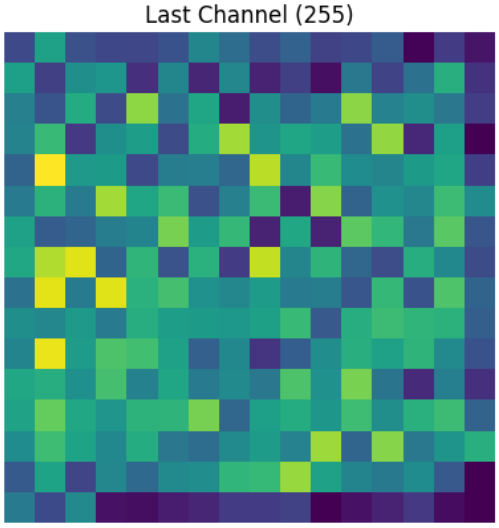

Figure 10: Channel 255 of laten representation

VII. BIBLIOGRAPHY

[1] M. L. a. M. F. B. Staar, "Anomaly detection with convolutional neural networks for industrial surface inspection," *Procedia CIRP*, vol. 79, p. 484–489, 2019.

[2] J. H. Y. F. Z. L. C. F. a. X. L. Q. Wang, " A comprehensive revisit to one class classification," in *2017 IEEE 2nd International Conference on Big Data Analysis (ICBDA)*, Beijing, 2017.

[3] A. B. a. V. K. V. Chandola, "Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 41, no. 3, p. 15, 28 May 2009.

[4] A. A. N. a. M. Turgeon, "A Comprehensive Study of Auto-Encoders for Anomaly Detection: Efficiency and Trade-Offs," *Machine Learning with Applications*, vol. 17, p. 10, 2024.

[5] F. Chollet, "Keras.io," 14 May 2016. [Online]. Available: https://blog.keras.io/building-autoencoders-in-keras.html. [Accessed 1 April 2025].

[6] GeeksforGeeks, "GeeksforGeeks," 11 March 2025. [Online]. Available: https://www.geeksforgeeks.org/implementing-an-autoencoder-in-pytorch/. [Accessed 28 March 2025].

[7] M. O. C. S. a. M. B. S. Dreiseitl, "Outlier Detection with One-Class SVMs: An Application to Melanoma Prognosis," *AMIA*, p. 172–176, 2010.

[8] A. T. O. Nizan, "arXiv.org," 28 May 2023. [Online]. Available: https://arxiv.org/abs/2305.17695. [Accessed 15 March 2025].

[9] I. o. E. a. Technology, "Wiley Online Library," [Online].

[10] "Artificial Intelligence for EM Problem Set - FIU," [Online]. Available: https://arc.fiu.edu/projects/artificial-intelligence-for-em-problem-set-dd/attachment/convolutional-autoencoder-cae-architecture/. [Accessed 1 April 2025].